

as-stock-information

as-stock-information 项目架构说明

本文与 stock-information-data 架构说明 同一格式，方便对照。as-stock-information 是 cms 平台资讯链路的对外接入层（AS = Anti-corruption / Aggregation Service），自身不持有数据，只做请求路由 + i18n + 访客判定。

如何查看本文档的图

含 Mermaid 图，启用方式：IDEA 装 *Mermaid* 插件、VSCode 装 *Markdown Preview Mermaid Support*、GitLab/GitHub 原生支持，或粘贴到 <https://mermaid.live> 临时预览。

1. 项目概述

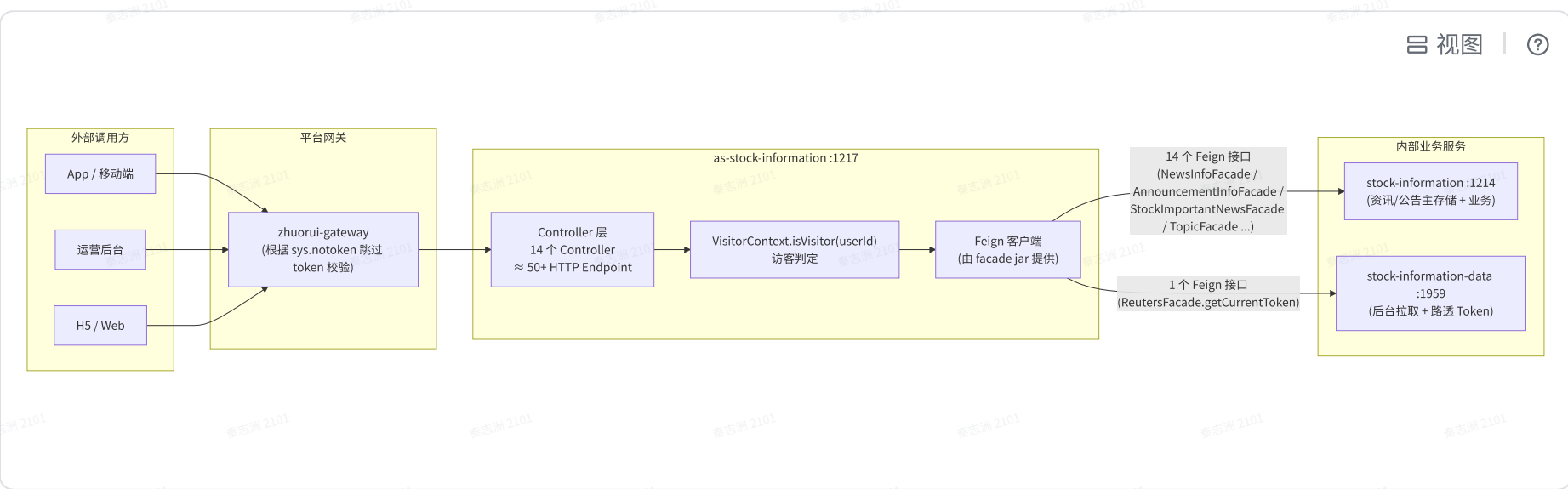
as-stock-information 是 cms 平台资讯类对外 API 网关，端口 1217，应用名 / Apollo app.id 都是 as-stock-information。在网关层（sys.notoken.server-name=as_stock_information）配置为免 token 服务，对外暴露给 APP 与运营后台。

核心职责（极简）：

- 接收外部 HTTP 请求（资讯列表、公告、卓锐洞察、要闻轮播、自选股资讯、路透 token 等）。
- 解析 userId / lang HTTP Header 做访客判定 + 多语言路由。
- 通过 Feign 把请求转发到内部业务服务：
 - stock-information（端口 1214）—— 14 个 Feign 接口，承担绝大多数业务。
 - stock-information-data（端口 1959）—— 仅 1 个 Feign 接口（路透 token 查询）。
- 完全不持有数据库、缓存、消息；不做任何持久化；只做 i18n 包装和返回结果包装。

启动类：AsInformationApplication.java，@EnableEurekaClient + @EnableFeignClients(basePackages={"com.zhuorui.*.facade"}) + @EnableHystrix + @EnableApolloConfig。

2. 数据流转

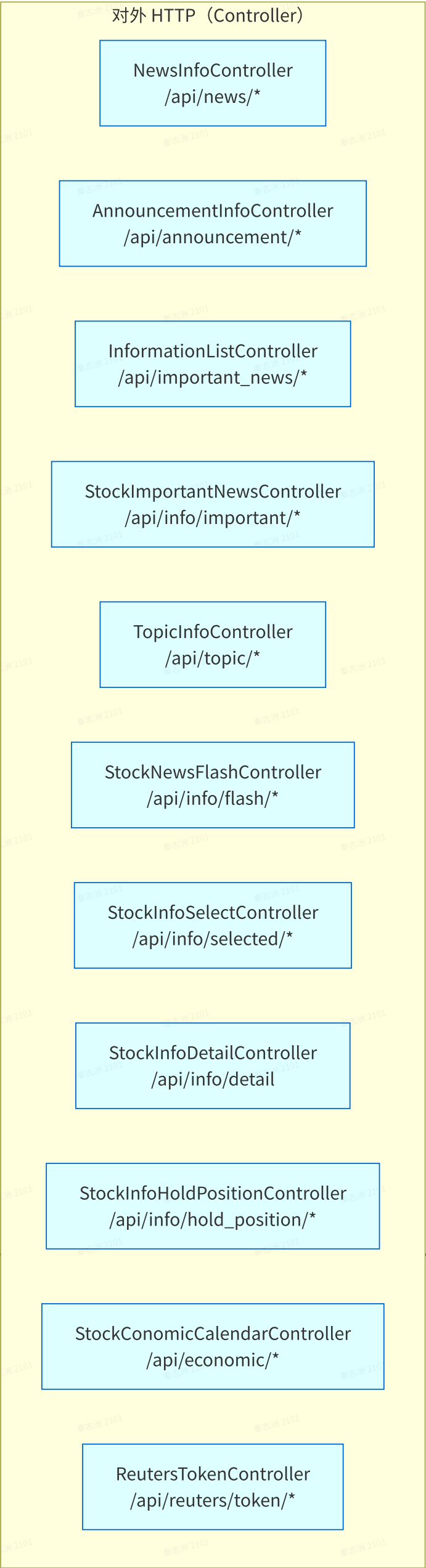


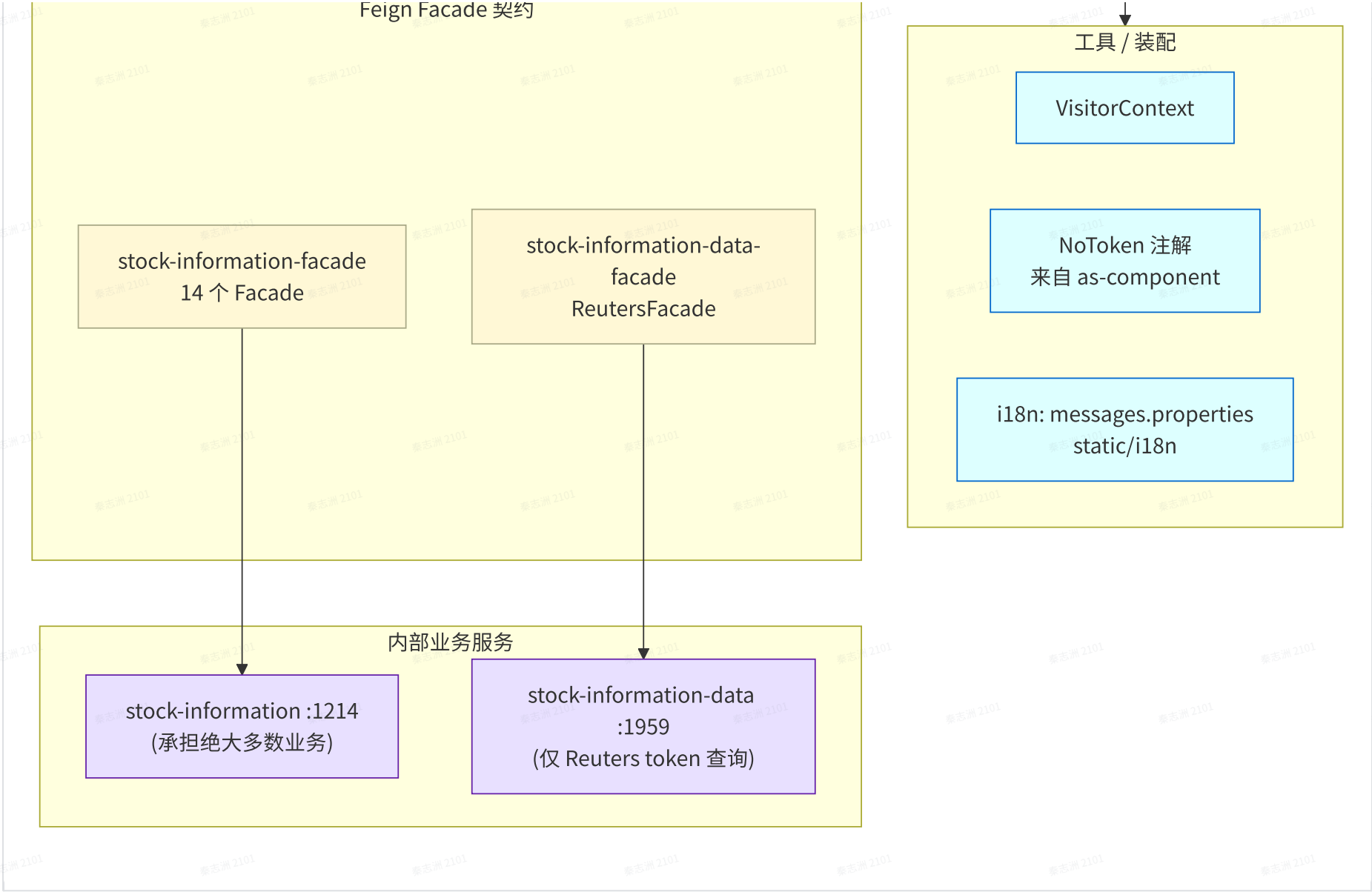
数据流的几个特点

点	说明
无数据库	pom.xml 不引入 mysql-component / mongo-component / redis-component /

	mq-component，只引入 springcloud-component + as-component + 两个 facade。
无缓存	没有 Caffeine / Redis 客户端，缓存逻辑全部在 stock-information 那一侧。
无 i18n 资源外置	spring.messages.basename=static/i18n/messages，多语言是把 lang Header 透传给后端服务，本服务自身基本不解析消息文案。
访客判定前置	VisitorContext.isVisitor(userId)：userId Header 为空 → 视为未登录访客；判定结果作为参数（notLogin / notLoginLang）透传给后端，让后端按访客口径返回数据（屏蔽某些字段或限制内容）。
@NoToken 注解	来自 as-component；标注后该接口在网关那一侧免 token 校验，访客也可直接调用。绝大多数接口都带这个注解。

3. 系统架构



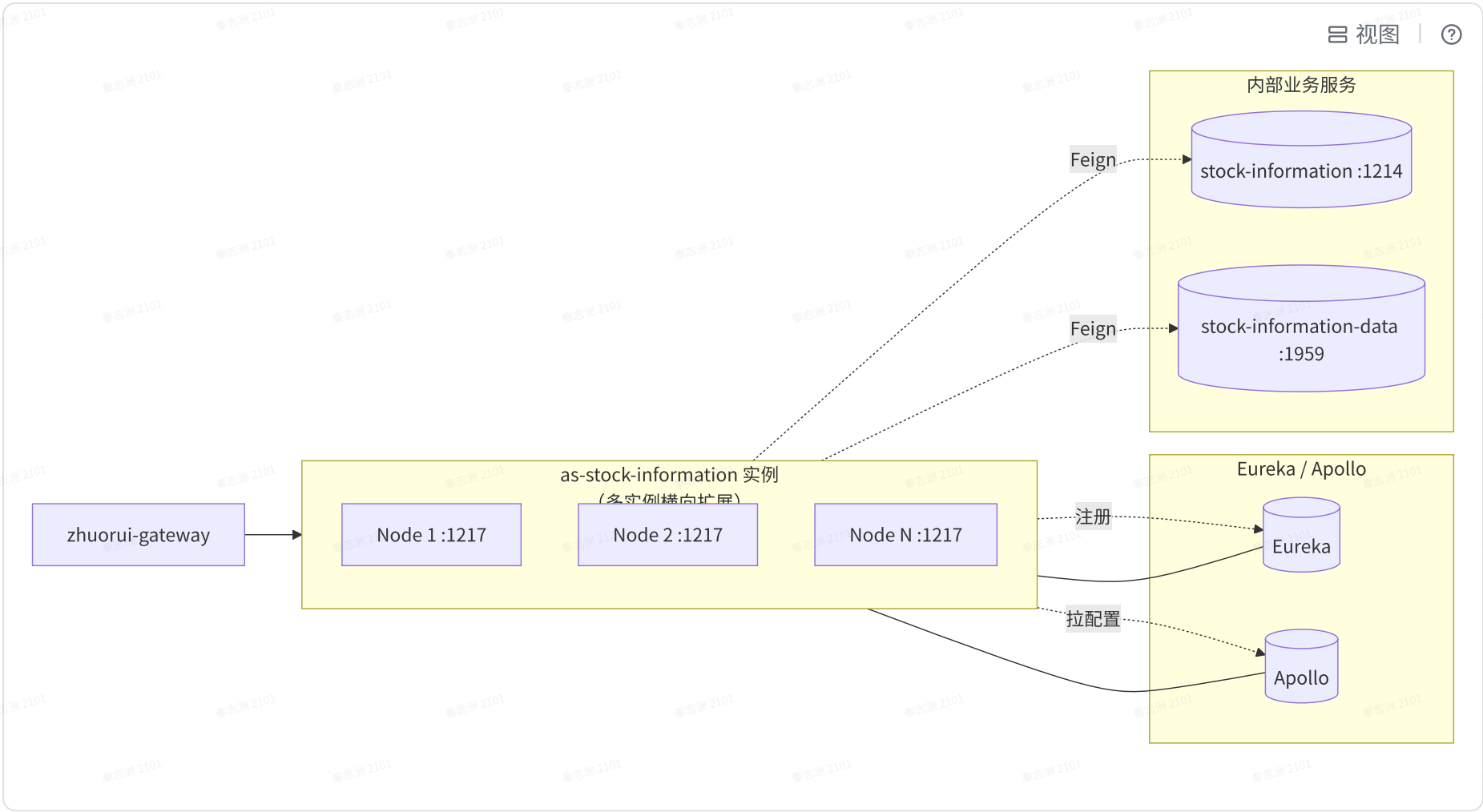


关键依赖 (pom.xml)

```
代码块
1  springcloud-component      # Eureka + Feign + Hystrix + Apollo 一揽子
2  as-component               # @NoToken / 通用过滤器 / i18n 工具
3  stock-information-facade    # 14 个 Feign (核心)
4  stock-information-data-facade # ReutersFacade
5  test-component             # TestNG (测试期)
```

依赖只有 5 个。这意味着该服务的 **build / 启动 / 内存占用都比 stock-information 轻量**，重启对线上风险极低。

4. 部署形态



配置要点 (bootstrap.yml)

- 默认端口 1217，应用名 `as-stock-information`，Apollo `app.id=as-stock-information`。
- Web 容器 Undertow: `io-threads=20`，`worker-threads=32`（与平台其他服务一致）。
- Hystrix: `timeoutInMilliseconds=9000`，`coreSize=100`，半隔离 `SEMAPHORE`，`maxConcurrentRequests=1000`。
- Ribbon: `ReadTimeout=3000ms`，`ConnectTimeout=1000ms`。
- `sys.notoken.server-name=as_stock_information`：网关层据此判定本服务为免 token 服务。
- Profile: `local / dev / pre / test / prod`。`prod` 仅注入 Eureka/Apollo 地址，业务相关配置全部通过 Apollo 下发。

部署上的好处：

1. **无状态**：可任意水平扩展；任何一个实例挂掉都无数据丢失。
2. **依赖少**：发布回滚速度快。
3. **故障域窄**：本服务故障不影响内部消费链路（`stock-information-data` 拉取/入库 / `notification` 推送等都不经过本服务）。

5. 业务范围

按 Controller 维度列出所有对外 HTTP 接口及其转发去向。`/api/...` 是该 Controller 路由前缀，单元粒度的接口路径按 Facade 接口对应。

5.0 APP 资讯页 ↔ AS 接口映射（最常用场景）

APP「资讯」一级 Tab 的页面结构（与原型图对照）：

代码块

- | | | | |
|---|------------|---|---------------|
| 1 | 顶部一级 Tab | ： 资讯 社区 视频 | ← APP 主导航 |
| 2 | 资讯页二级 Tab | ： 自选 卓锐洞察 要闻 快讯 加密货币 新股 宏观 [= 更多] | |
| 3 | 二级 Tab 下筛选 | ： 新闻 ▾ 所有公告 ▾ 评级 | ← 内容类型筛选 chip |

整个资讯页只用 4 个核心接口——这是本服务流量的 80%+。其它 Controller 仍提供的接口（要闻独立链路、自选 selected、个股详情等）属于历史 / 个股详情页 / 其他端入口，不参与当前 APP 主资讯页：

APP UI 元素	AS HTTP 接口	AS Controller	AS Facade（转发目标）
Tab：自选 / 卓锐洞察 / 要闻 / 加密货币 / 新股 / 宏观（及 ≡ 展开的港股 / 美股 / 异动等）→ chip「新闻」「评级」	POST /api/news/v1/list	NewsInfoController	NewsInfoFacade.getList
Tab：快讯	POST /api/news/v1/fast_list	NewsInfoController	NewsInfoFacade.pullFastNewsList
Tab：自选 → chip「所有公告」	POST /api/announcement/v1/list	AnnouncementInfoController	AnnouncementInfoFacade.announcementList
Tab：卓锐洞察 → 顶部首推（banner + hot）	POST /api/news/v1/get_recommend	NewsInfoController	NewsInfoFacade.getRecommend

资讯页的主列表只靠 /api/news/v1/list（含卓锐洞察 Tab，传 type=14 即 ZHUORUI_LOOKS_AT_THE_MARKET 卓锐看市）+ /api/news/v1/fast_list（快讯专 Tab）+ /api/announcement/v1/list（公告 chip）这 3 个接口。/api/news/v1/get_recommend 只是卓锐洞察 Tab 顶部"首推"模块的副接口（服务端帮 APP 预置 isRecommend=1 + type=14，并按 recommended_bit 拆 banner / hot），本质上仍是同一份 NewsDao.pullNewsList。

列表项点击进入详情 / AI 补全：

行为	AS HTTP 接口	转发目标
点列表项进详情页	POST /api/news/v1/content	NewsInfoFacade.content
详情页底部「AI 助手相关资讯」	POST /api/news/v1/ai_make_up	NewsInfoFacade.aiMakeUpNews
列表项关联个股的实时价格（如截图里 谷歌-C 397.050 +0.44%）	不走本服务	APP 拿到 news.codes 后单独调 stock-market-* 服务刷价格

关键约定

1. 要闻 Tab 不再走 Redis ZSet 独立链路：当前 APP「要闻」二级 Tab 就是普通 Tab，与加密货币 / 新股 / 宏观 一样走 /api/news/v1/list，靠 type=MAJOR_NEWS(0) 区分。InformationListController (/api/important_news/*) 与 StockInfoSelectController (/api/info/selected/*) 仍存在但当前主资讯页未使用，按历史 / 其他端入口对待。
2. 筛选维度全部塞进请求体：Tab → type（卓锐洞察=14、要闻=0、加密货币=15、新股=6、宏观=11、港股=1、美股=2、异动=5、AI 看盘=7）；自选 Tab → selectedCodes（用户自选股 ts_code 列表）；右上「全部 ▾」市场筛选 → market（1 港股 / 2 美股 / 3 沪深）；chip「评级」→ type=RATING(10)；chip「新闻」→ type=NEWS(8) 或不传 type；chip「所有公告」→ 切到 /api/announcement/v1/list。

3. 「卓锐洞察 Tab」分两块：上半部分首推（banner+hot）调 `/api/news/v1/get_recommend`（服务端预置 `isRecommend=1 + type=14`）；下半部分常规列表与其它 Tab 一样调 `/api/news/v1/list + type=14`，只是不带 `isRecommend` 过滤。
4. 「卓锐洞察 Tab」与「卓锐洞察推送」是两件事：Tab 读 Mongo `t_news` 中 `type=14`（卓锐看市）的资讯，拉取式；后台运营手工发起的「卓锐洞察推送」走 SI 的 `news_push_centre` 表 + `PushNewsHandler.PUSH_NEWS_ZY` 定时塞通知中心，与 Tab 完全两条路径（详见 stock-information-data 架构 § 4.4）。
5. 自选 Tab 需登录：访客（`VisitorContext.isVisitor(userId)==true`）情况下 `notLogin=true` 透传到 SI；自选 Tab 因没有 `selectedCodes` 通常前端会拦截不发请求；其余 Tab 全部带 `@NoToken`，访客可直接看（部分付费内容如路透资讯由 `notLogin=true` 在 DAO 层过滤）。
6. 下拉刷新 / 上拉加载：以请求体里的 `pubTime` / `newsId` 作为游标，不靠 path 区分 last/old —— 与历史 `/api/important_news/v1/all_info_list_new` / `_old` 那套两路接口不同。

5.1 Controller 一览

#	Controller	URL 前缀	转发到的 Facade	主资讯页	用途
1	NewsInfoController	/api/news	NewsInfoFacade + StockImportantNewsFacade	✓	资讯页主力： list（7 Tab+chip 共用主列表，卓锐洞察传 <code>type=14</code> ） / fast_list（快讯 Tab）； get_recommend 仅卓锐洞察 Tab 顶部首推副接口； 外加 content / ai_make_up / get_carousel_list / topic_list / related_to pic_news
2	AnnouncementInfoController	/api/announcement	AnnouncementInfoFacade	✓	资讯页公告 chip: /v1/list（带 <code>selectedCodes</code> ），及详情 /v1/content、个股详情页的 /v1/important_list
3	InformationListController	/api/important_news	AllInformationListFacade	✗（历史）	早期「要闻」Tab 独立

			code		Redis ZSet 链路 (all_info_list_new/_old)； 当前主资讯页改走 /api/news/v1/list， 但接口保留以兼容老端
4	StockImportantNewsController	/api/info/important	StockImportantNewsFacade	✗	个股详情页的"重要资讯"模块（按ts/code取该股的要闻）
5	TopicInfoController	/api/topic	TopicFacade + ConTopicFacade	✗	主题列表 / 主题详情 / 主题分类（如"风险早知道"、"机会风向标"）
6	StockNewsFlashController	/api/info/flash	StockNewsFlashFacade	✗	7×24 快讯流（独立快讯页 / 加密货币快讯页备用）
7	StockInfoSelectController	/api/info/selected	StockInfoSelectFacade	✗（历史）	早期自选 Tab 专用 get_last / get_old； 当前自选 Tab 改走 /api/news/v1/list?selectedCodes=...， 本接口保留
8	StockInfoDetailController	/api/info	StockInfoDetailFacade	✗	个股详情页的资讯聚合（资讯 + 公告 + 异动一并返回）
9	StockInfoHoldPositionController	/api/info/hold_position	StockInfoHoldPositionFacade	✗	持仓页的相关资讯（按持仓股票拉资讯）
10	StockConomicCalendarController	/api/economic	StockConomicCalendarFacade	✗	经济日历（财经事件日程）
11	ReutersTokenController	/api/reuters/token	ReutersFacade（来自 stock-information-	✗	路透 Token 查询（不是 APP 用，是给富文本组件等内部调用）

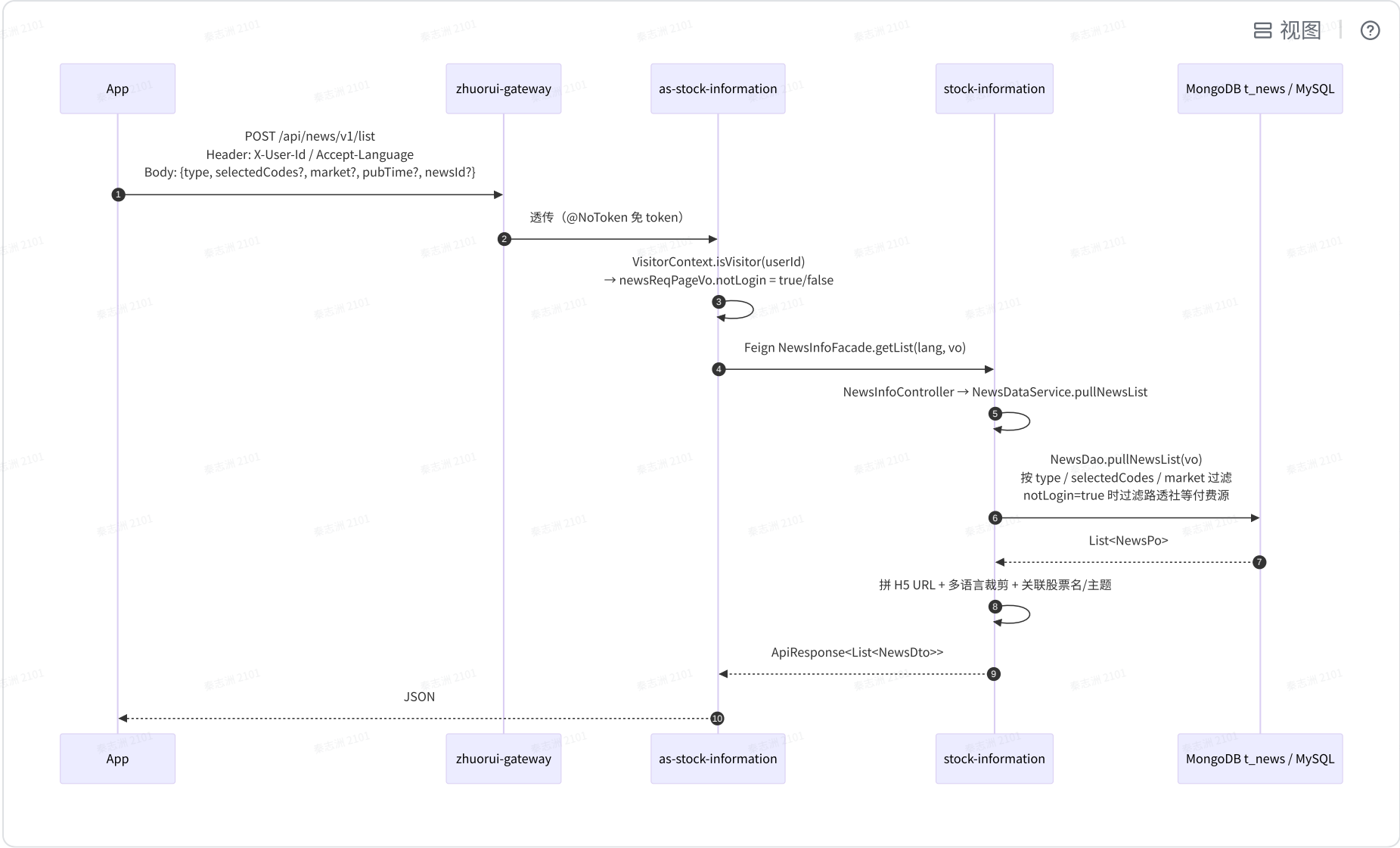
			data-facade)		
--	--	--	--------------	--	--

当前 APP 资讯主页（原型图所示）只调 #1 + #2 两个 Controller。#3 / #7 是同一页的历史链路，保留但不再被主流量打到。

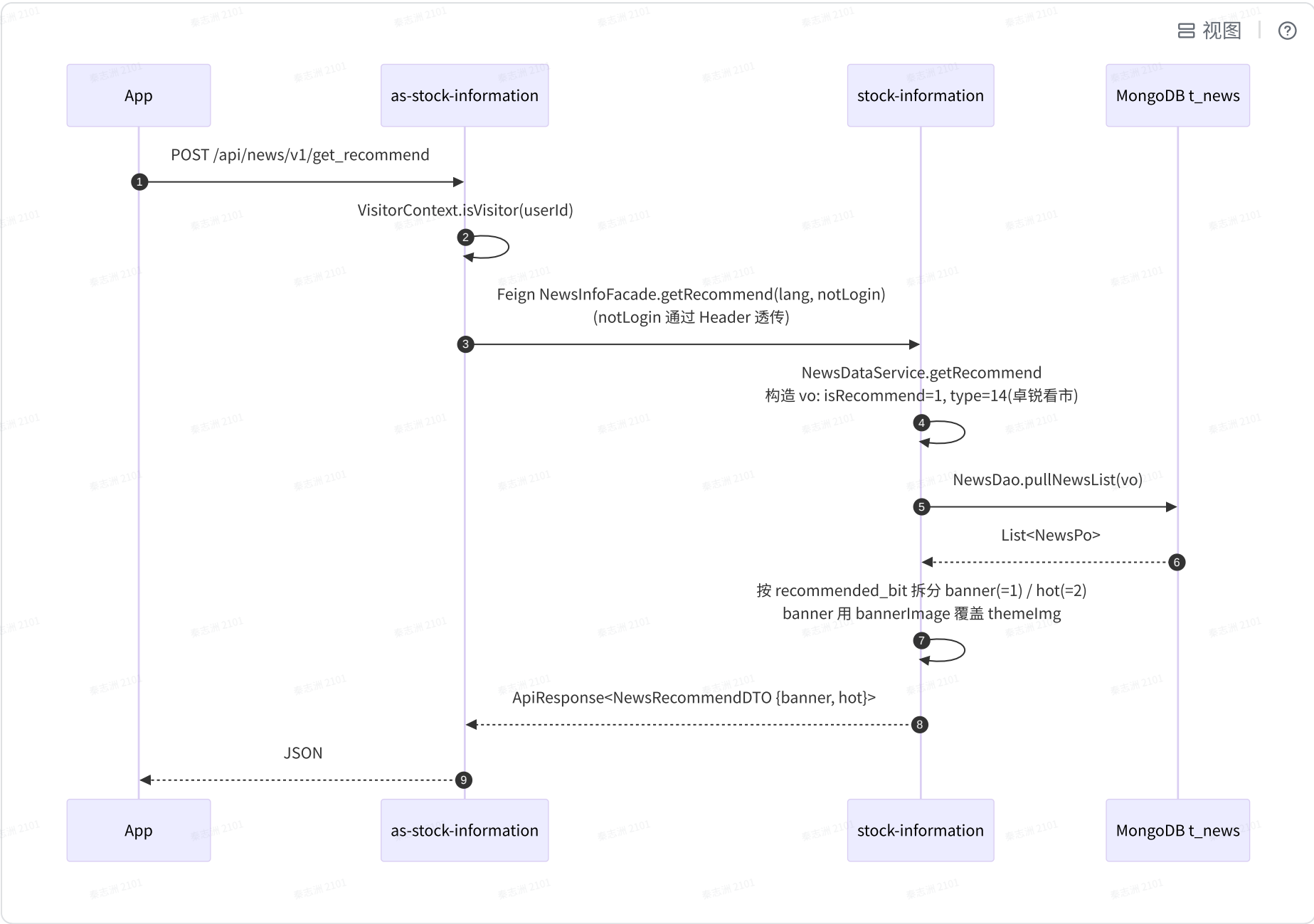
5.2 4 个核心接口的请求转发链路

A / B / C / D 是 APP 资讯主页的全部出口；其中 A 单接口承担 7 个 Tab 的全部主列表流量（含卓锐洞察 Tab 的下半部分常规列表），B 只是卓锐洞察 Tab 顶部"首推"模块的副接口。

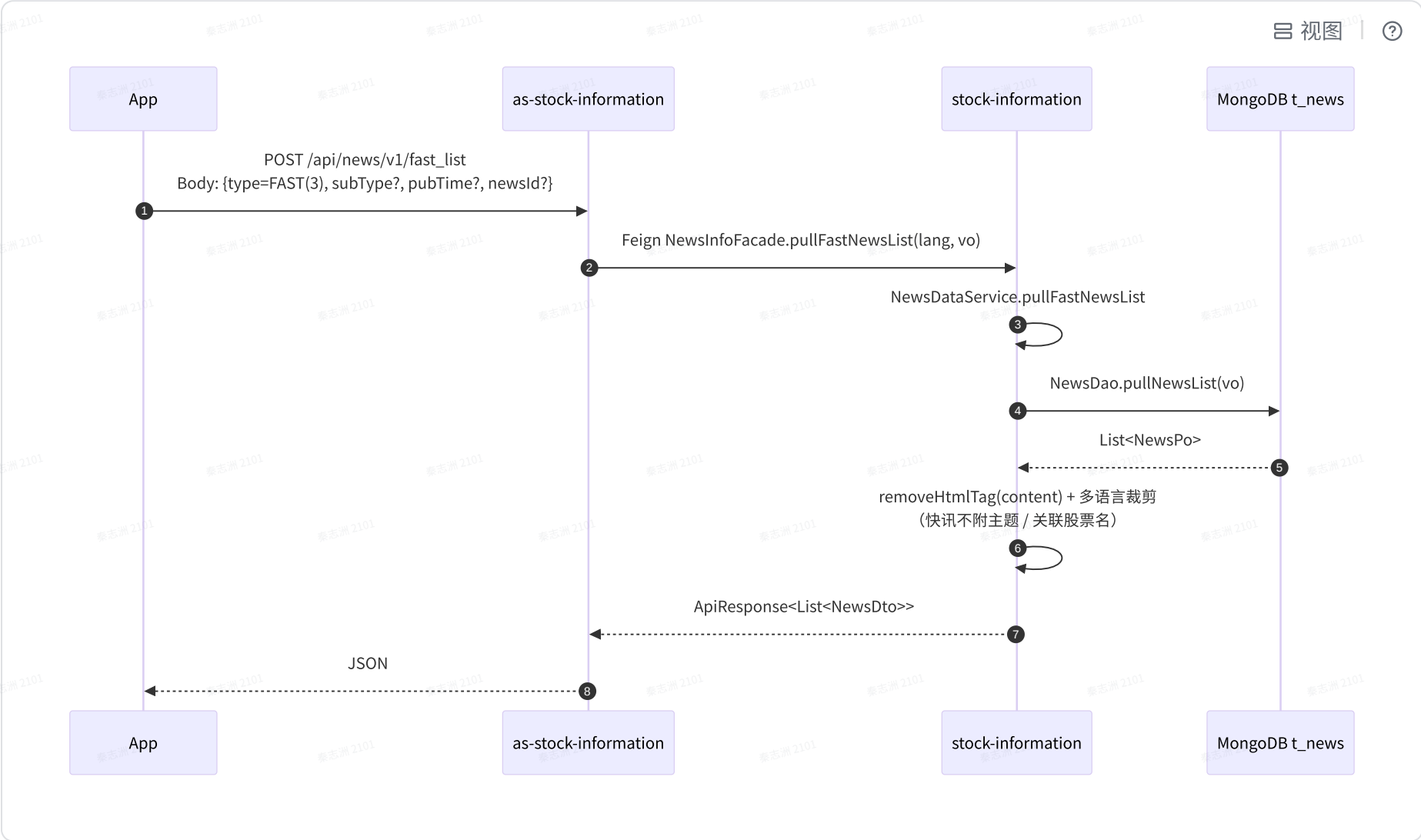
A. 通用资讯列表（POST /api/news/v1/list）—— 自选 / 卓锐洞察 / 要闻 / 加密货币 / 新股 / 宏观 等共用



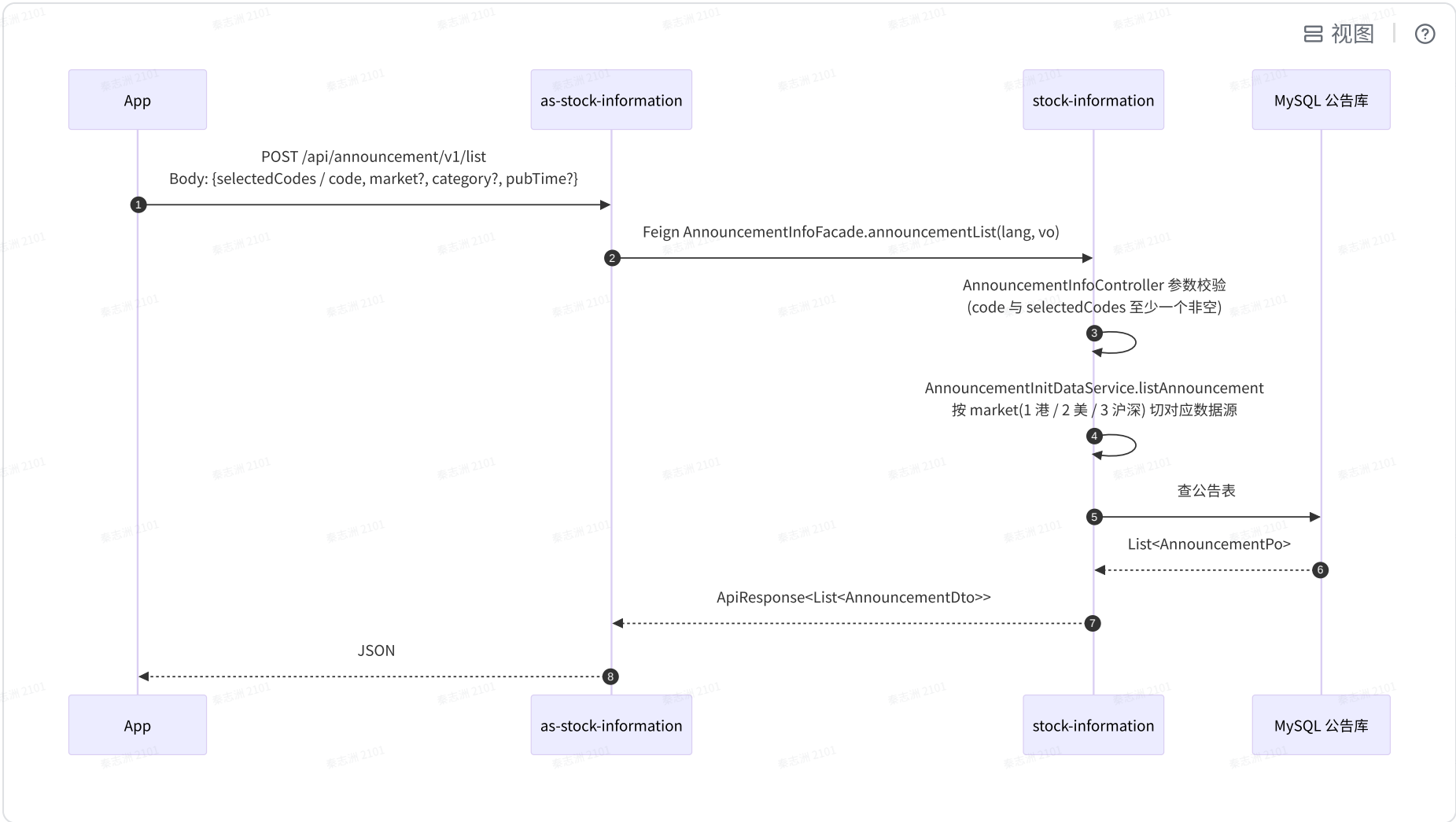
B. 卓锐洞察 Tab 顶部首推（POST /api/news/v1/get_recommend）—— 副接口，Tab 主列表仍走 A



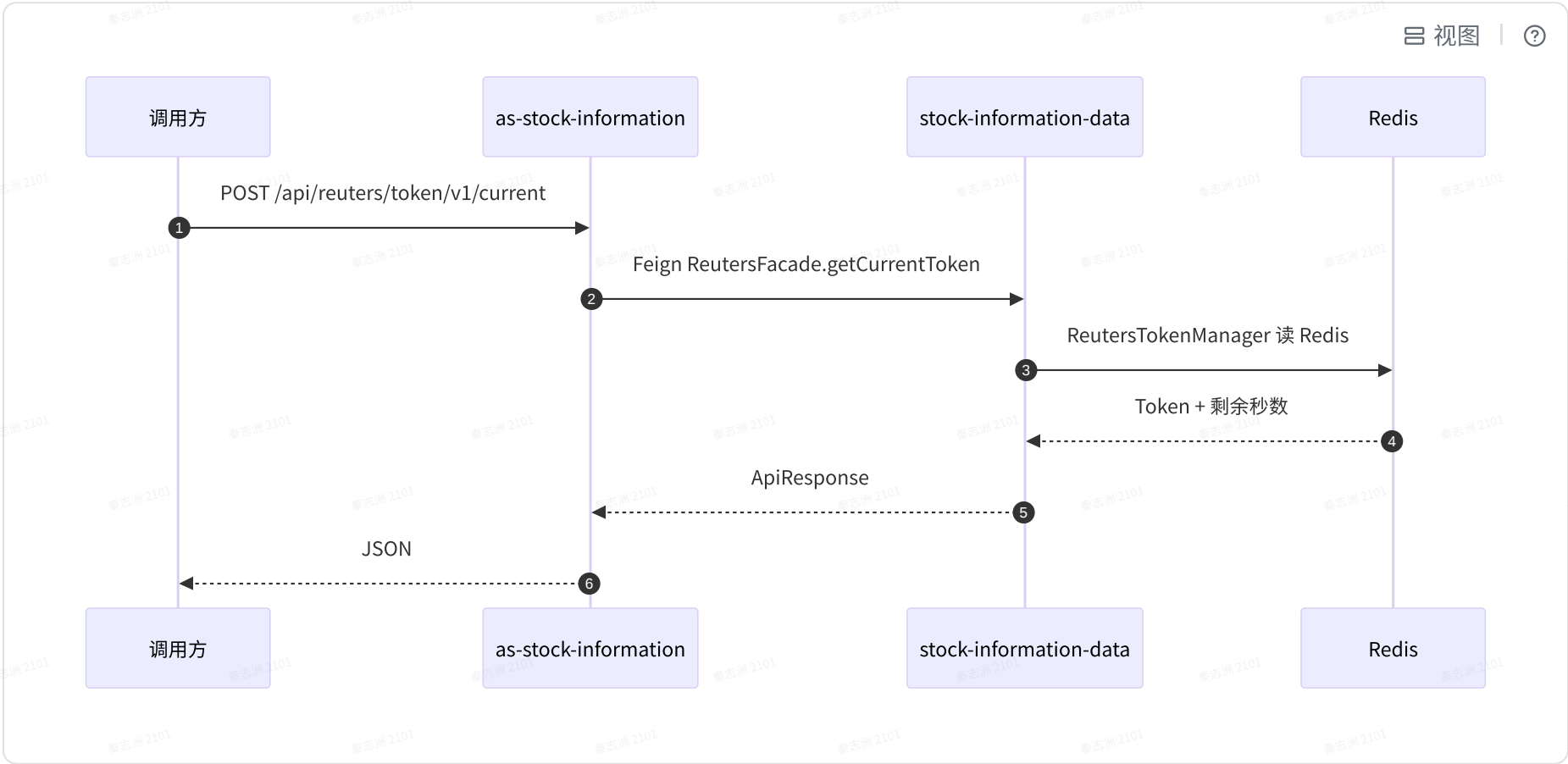
C. 快讯 (POST /api/news/v1/fast_list)



D. 自选股公告 (POST /api/announcement/v1/list) —— 自选 Tab 下「所有公告」chip



E. 路透 Token 查询（ POST /api/reuters/token/v1/current ）—— 唯一指向 -data



E 是给富文本组件等内部用，不是 APP 直接调。AS 本身只做 i18n + 访客判定，没有缓存 / 持久化 / 业务判断。

5.3 访客与登录态如何流到后端

- 请求侧： X-User-Id Header 由网关注入（已登录用户）或为空（访客）。
- AS 侧： VisitorContext.isVisitor(userId) (VisitorContext.java) 做唯一判定。
- 下游侧：把判定结果写到 XxxReqVo.notLogin （请求体字段）或 notLogin Header 一并发给 stock-information，后端服务不再做登录态判定，只看 notLogin 控制返回粒度。

这种 "访客判定只在 AS 边界发生" 的设计，使后端服务可以保持纯业务，未登录/已登录返回的差异统一收敛在 AS 入口处可控。

5.4 没有的能力（与 stock-information / -data 对比）

能力	as-stock-information
数据库读写	✗
Redis 缓存读写	✗
MongoDB 读写	✗
Kafka / RabbitMQ 生产消费	✗
XXL-Job 定时任务	✗
资讯抓取 / 公告抓取	✗
业务实体（PO）	✗
多数据源切面	✗
OSS 上传	✗

6. 代码模块

代码块

```
1  as-stock-information/
2  |— pom.xml                      # 仅 5 个依赖
3  |— src/main/java/com/zhuorui/asstockinformation/
4  |   |— AsInformationApplication.java    # 启动类
5  |   |— config/
6  |   |   |— package-info.java          # 占位（无实际配置类）
7  |   |— controller/                   # 11 个 Controller，全部 @RestController
8  |   |   |— NewsInfoController.java
9  |   |   |— AnnouncementInfoController.java
10  |   |   |— InformationListController.java
11  |   |   |— StockImportantNewsController.java
12  |   |   |— TopicInfoController.java
13  |   |   |— StockNewsFlashController.java
14  |   |   |— StockInfoSelectController.java
15  |   |   |— StockInfoDetailController.java
16  |   |   |— StockInfoHoldPositionController.java
17  |   |   |— StockConomicCalendarController.java
18  |   |   |— ReutersTokenController.java
19  |   |— util/
20  |   |   |— VisitorContext.java        # 唯一工具类，访客判定
21  |— src/main/resources/
22  |   |— bootstrap.yml + bootstrap-{profile}.yml
23  |   |— log4j2-boot-{profile}.xml
```

设计上的约定

- **Controller 只做转发：** `@RestController` 接收 HTTP → 调一个 `*Facade` → 返回。任何业务判断（如"该用户是否能看这条 VIP 资讯"）都在后端，不在这里。
- **不持有 Service：** `controller/service/dao` 三层结构里的下两层在 AS 不存在。
- **i18n 透传：** `Accept-Language` / `lang` Header 解析为字符串透传给后端，AS 自己不维护语言资源（`spring.messages.basename` 是 spring 默认值，没有真正使用）。

- 未来新增接口的步骤：
 - a. 在 stock-information-facade （或 -data-facade）里加 @FeignClient 方法；
 - b. 在 stock-information （或 -data）里实现对应 @RestController ；
 - c. 在 AS 这边加一个薄 controller 方法，把 lang / userId 透传过去；
 - d. 加 @NoToken 如果走访客流量。

7. 实体 ER 图

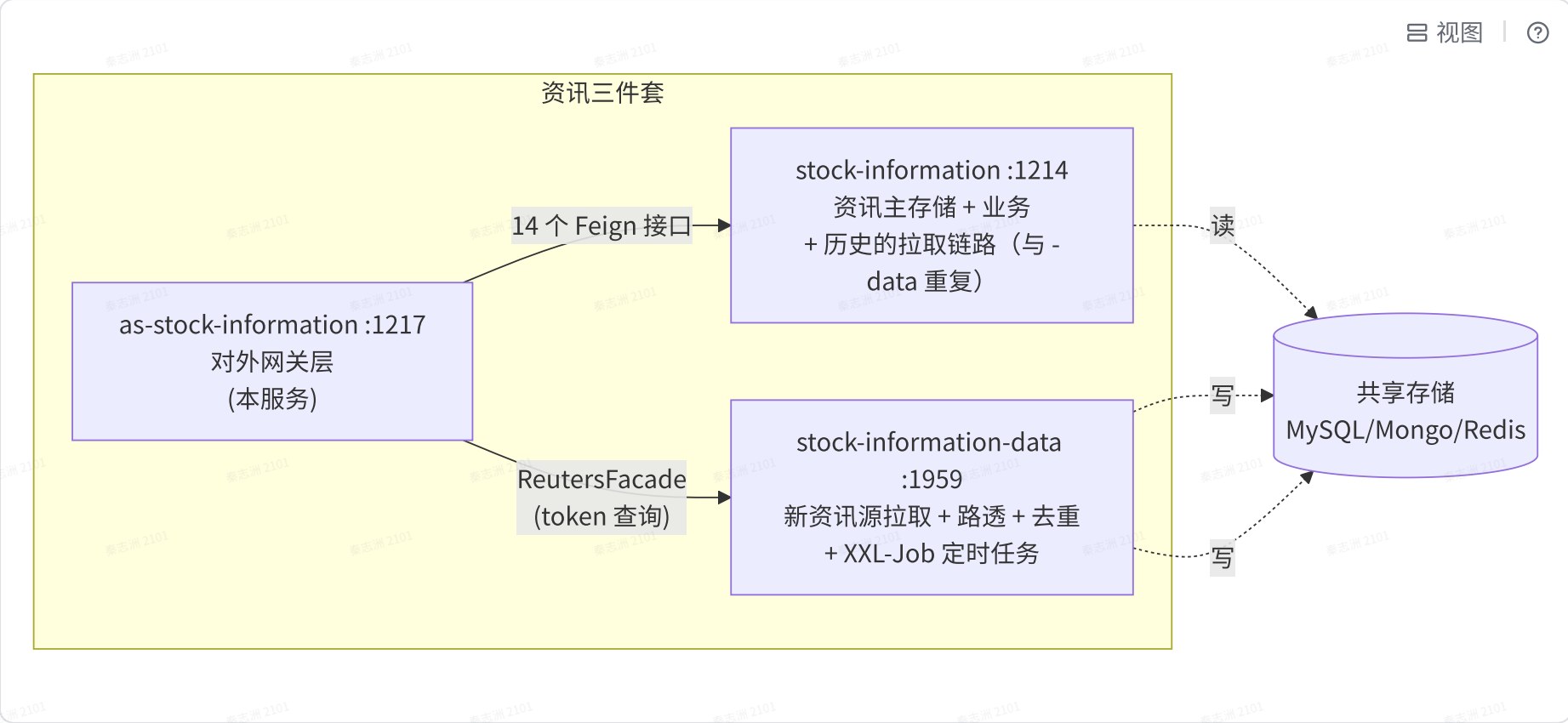
本服务无实体——所有 PO / Mongo 文档 / Redis Key 都属于 stock-information 或 stock-information-data。详细 ER 图请参考：

- stock-information-data 架构说明 § 7 实体 ER 图

AS 接收的请求体 / 响应体 DTO 都来自 stock-information-facade jar：

DTO 类（来自 facade jar）	用途
NewsDto / MakeUpNewsDto / NewsRecommendDTO	资讯返回体
NewsReqPageVo / NewsReqVo	资讯请求体
AnnouncementDto / AnnouncementReqVo	公告请求/返回
ImportantBannerDto	要闻轮播图
StockBaseInformationDto / InfoListTypeRequestVo	自选股资讯
InfoDetailDto （来自 model-component）	个股资讯详情

8. 与兄弟服务的关系



职责切分（与本仓库其他两个服务对照）：

维度	as-stock-information	stock-information	stock-information-data
端口	1217	1214	1959
主要角色	对外 API 网关	对内业务服务（被 Feign 调）	后台拉取/批处理（XXL-Job）
数据库	无	11 个数据源	11 个数据源（与 SI 同库）
Mongo / Redis	无	读写	读写
Kafka / RabbitMQ	无	RabbitMQ 消费/生产	Kafka + RabbitMQ
抓取定时任务	无	有（与 -data 重复，历史遗留）	有（新资讯源主战场，含路透）
是否对外暴露 HTTP	是（经网关）	否（内部 Feign）	否（仅内部 Feign）

注：stock-information 与 stock-information-data 的 xxljobhandler/ scheduled/ service/ po/ 高度重复，是因为 -data 是从 -information 拷贝出来另起的服务。当前实际生产中新接资讯源（路透 / CoinDesk / FiNet / Techub 等）只在 -data 跑，老资讯源（智通 / 格隆汇 / 凤凰网 / BlockBeats / 公告）的 XXL-Job 在哪一侧执行需要看运维端的 cron 注册情况。AS 不感知这层差异，永远只调 stock-information 的对外 API。

9. 维护提示

- 加接口：先在 stock-information-facade 里加 @FeignClient 方法 + 在 stock-information 里实现 controller，再在 AS 加一个一行的 controller 方法转发。
- 改 i18n：本服务不持有语言资源，去 stock-information 那边改 messages_*.properties。
- 故障排查：AS 报错通常是下游 Feign 调用超时（Hystrix timeoutInMilliseconds=9000） / 反序列化失败 / 网关层 token 配置变更；本服务自身极少出独立 bug。
- 本地启动：mvn spring-boot:run -pl as-stock-information -Dspring-boot.run.profiles=local，需要可达的 Apollo + Eureka + 至少一个 stock-information 实例。

生成日期：2026-05-09 · 修订：2026-05-11（按 APP 原型图收敛主资讯页为 4 接口）